

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины
«Искусственный интеллект и машинное обучение»**

Выполнил:
Лебединский Даниил
Александрович 2 курс,
группа ИВТ-б-о-24-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Ссылка на репозиторий:

<https://github.com/FomalhautSystem/AI-LAB-4/tree/main>

Ход работы:

1. Создание Series из списка.

Функция `pd.Series([1, 2, 3, 4, 5])` создаёт одномерный массив с целочисленным индексом по умолчанию (0, 1, 2, ...). Метод `print()` выводит значения с указанием типа данных `dtype: int64`.

Листинг:

```
s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)
```

Результат выполнения:

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

2. Создание Series с пользовательским индексом.

При создании `pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])` второй аргумент задаёт метки индекса. Это позволяет обращаться к элементам по строковым ключам вместо числовых позиций.

Листинг:

```
s2 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
print(s2)
```

Результат выполнения:

a 1

b 2

c 3

d 4

e 5

dtype: int64

3. Создание Series из NumPy массива.

Функция `np.array([1, 2, 3, 4, 5])` создаёт массив NumPy, который затем передаётся в `pd.Series()` вместе с пользовательским индексом. Это демонстрирует совместимость библиотек.

Листинг:

```
ndarr = np.array([1, 2, 3, 4, 5])
s3 = pd.Series(ndarr, ['a', 'b', 'c', 'd', 'e'])
print(s3)
```

Результат выполнения:

a 1

b 2

c 3

d 4

e 5

dtype: int64

4. Создание Series из словаря.

Словарь `{'a':1, 'b':2, 'c':3}` автоматически преобразуется в Series: ключи становятся индексом, значения – данными. Порядок элементов сохраняется.

Листинг:

```
d = {'a':1, 'b':2, 'c':3}
s4 = pd.Series(d)
```

```
print(s4)
```

Результат выполнения:

```
a    1
```

```
b    2
```

```
c    3
```

```
dtype: int64
```

5. Создание Series со скалярным значением.

При передаче скаляра $a = 7$ и списка индексов ['a', 'b', 'c'] значение 7 умножается для всех указанных меток индекса.

Листинг:

```
a = 7
```

```
s5 = pd.Series(a, ['a', 'b', 'c'])
```

```
print(s5)
```

Результат выполнения:

```
a    7
```

```
b    7
```

```
c    7
```

```
dtype: int64
```

6. Доступ к элементам: по метке и по позиции.

`s6['c']` – доступ по метке индекса, `s6.iloc[2]` – доступ по целочисленной позиции. Оба способа возвращают значение 3.

Листинг:

```
s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
```

```
print(s6['c'])
```

```
print(s6.iloc[2])
```

Результат выполнения:

```
3
```

3

7. Срез по меткам индекса.

`s6[:2]` возвращает первые два элемента с метками 'a' и 'b'. Срезы в Pandas работают аналогично спискам.

Листинг:

```
print(s6[:2])
```

Результат выполнения:

```
a    1
```

```
b    2
```

```
dtype: int64
```

8. Булева индексация.

Выражение `s6[s6 <= 3]` возвращает только те элементы, значения которых меньше или равны 3. Это мощный инструмент фильтрации данных.

Листинг:

```
print(s6[s6 <= 3])
```

Результат выполнения:

```
a    1
```

```
b    2
```

```
c    3
```

```
dtype: int64
```

9. Арифметические операции с Series.

При сложении `s6 + s7` элементы с одинаковыми метками индекса суммируются. Умножение на скаляр `s6 * 3` применяется поэлементно ко всем значениям.

Листинг:

```
s7 = pd.Series([10, 20, 30, 40, 50], ['a', 'b', 'c', 'd', 'e'])
```

```
print(s6 + s7)
```

```
print(s6 * 3)
```

Результат выполнения:

a 11

b 22

c 33

d 44

e 55

dtype: int64

a 3

b 6

c 9

d 12

e 15

dtype: int64

10. Доступ по позиции: `iloc`.

Метод `iloc` позволяет обращаться к элементам по целочисленным позициям: `iloc[0]` – первый элемент, `iloc[-1]` – последний.

Листинг:

```
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
```

```
print(s.iloc[0])
```

```
print(s.iloc[2])
```

```
print(s.iloc[-1])
```

Результат выполнения:

10

30

50

11. Доступ по меткам: `loc`.

Метод `loc` работает с метками индекса: `s.loc['b':'d']` возвращает срез от 'b' до 'd' включительно.

Листинг:

```
print(s.iloc[1:3])  
print(s.loc['b':'d'])
```

Результат выполнения:

```
b    20  
c    30  
dtype: int64  
b    20  
c    30  
d    40  
dtype: int64
```

12. Фильтрация по условию.

Выражение `s[s > 10]` возвращает только элементы, значения которых больше 10. Булев массив используется как маска для выбора.

Листинг:

```
s = pd.Series([10,25,8,30,15], index=['a','b','c','d','e'])  
filtered_s = s[s > 10]  
print(filtered_s)
```

Результат выполнения:

```
b    25  
d    30  
e    15  
dtype: int64
```

13. Создание булева массива.

Выражение `s > 10` возвращает Series с булевыми значениями, показывая, для каких элементов условие истинно.

Листинг:

```
print(s > 10)
```

Результат выполнения:

a False

b True

c False

d True

e True

dtype: bool

14. Составное условие.

Оператор & позволяет комбинировать условия: $(s \geq 10) \& (s \leq 30)$ выбирает элементы в диапазоне [10, 30].

Листинг:

```
filtered_s = s[(s >= 10) & (s <= 30)]  
print(filtered_s)
```

Результат выполнения:

a 10

b 25

d 30

e 15

dtype: int64

15. Фильтрация по списку меток.

Метод `isin(['b','d'])` проверяет принадлежность меток индекса заданному списку и возвращает соответствующие элементы.

Листинг:

```
filtered_s = s[s.index.isin(['b','d'])]  
print(filtered_s)
```

Результат выполнения:

b 25


```
d    30
```

```
dtype: int64
```

16. Работа с пропущенными значениями: `notnull()`.

Метод `notnull()` возвращает булев массив, где `True` соответствует непустым значениям. Используется для фильтрации.

Листинг:

```
s_with_nan = pd.Series([10, None, 8, 30, None], index=['a', 'b', 'c', 'd', 'e'])
```

```
filtered_s = s_with_nan[s_with_nan.notnull()]
```

```
print(filtered_s)
```

Результат выполнения:

```
a    10.0
```

```
c     8.0
```

```
d    30.0
```

```
dtype: float64
```

17. Модификация значения по метке.

`s.loc['b'] = 25` изменяет значение элемента с меткой 'b' на 25. Это пример прямого присваивания.

Листинг:

```
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
s.loc['b'] = 25
```

```
print(s)
```

Результат выполнения:

```
a    10
```

```
b    25
```

```
c    30
```

```
d    40
```

```
dtype: int64
```

18. Условное присваивание.

Выражение `s[s > 30] += 10` увеличивает на 10 только те элементы, которые больше 30.

Листинг:

```
s = pd.Series([10,20,30,40], index=['a','b','c','d'])
s[s > 30] += 10
print(s)
```

Результат выполнения:

```
a    10
b    20
c    30
d    50
dtype: int64
```

19. Присваивание нескольким элементам.

`s.loc[['a','c']] = [100, 200]` одновременно изменяет значения элементов с метками 'a' и 'c'.

Листинг:

```
s.loc[['a','c']] = [100, 200]
print(s)
```

Результат выполнения:

```
a    100
b     20
c    200
d     50
dtype: int64
```

20. Применение функции: `apply()`.

Метод `apply(lambda x: x2)` применяет лямбда-функцию возведения в квадрат ко всем элементам Series.

Листинг:

```
s = s.apply(lambda x: x2)
print(s)
```

Результат выполнения:

```
a    200
b     40
c    400
d    100
dtype: int64
```

21. Заполнение пропущенных значений.

Метод `fillna(0)` заменяет все значения NaN на 0. Полезно для предобработки данных.

Листинг:

```
s_with_nan = pd.Series([10, None, 8, 30, None], index=['a', 'b', 'c', 'd', 'e'])
s_filled = s_with_nan.fillna(0)
print(s_filled)
```

Результат выполнения:

```
a    10.0
b     0.0
c     8.0
d    30.0
e     0.0
dtype: float64
```

22. Просмотр первых элементов: `head()`.

Метод `head(3)` возвращает первые 3 элемента, `head()` без аргумента – первые 5 элементов по умолчанию.

Листинг:

```
s = pd.Series([10, 20, 30, 40, 50, 60, 70], index=['a', 'b', 'c', 'd', 'e', 'f', 'g'])
print(s.head(3))
```

```
print(s.head())
```

Результат выполнения:

```
a    10
```

```
b    20
```

```
c    30
```

```
dtype: int64
```

```
a    10
```

```
b    20
```

```
c    30
```

```
d    40
```

```
e    50
```

```
dtype: int64
```

23. Просмотр последних элементов: `tail()`.

Метод `tail(3)` возвращает последние 3 элемента, `tail()` без аргумента – последние 5.

Листинг:

```
print(s.tail(3))
```

```
print(s.tail())
```

Результат выполнения:

```
e    50
```

```
f    60
```

```
g    70
```

```
dtype: int64
```

```
c    30
```

```
d    40
```

```
e    50
```

```
f    60
```

```
g    70
```

dtype: int64

24. Работа с индексом.

Атрибут `s.index` возвращает объект индекса. Доступ к элементам индекса осуществляется как к списку: `s.index[0]`, `s.index[-1]`.

Листинг:

```
s = pd.Series([10,20,30,40,50], index=['a','b','c','d','e'])
print(s.index)
print(s.index[0])
print(s.index[-1])
```

Результат выполнения:

```
Index(['a', 'b', 'c', 'd', 'e'], dtype='str')
a
e
```

25. Доступ к значениям: `values`.

Атрибут `s.values` возвращает массив значений типа `numpy.ndarray`. К элементам можно обращаться по индексу.

Листинг:

```
print(s.values)
print(s.values[0])
print(s.values.mean())
```

Результат выполнения:

```
[10 20 30 40 50]
10
30.0
```

26. Тип данных: `dtype`.

Атрибут `dtype` показывает тип данных элементов: `int64`, `float64`, `str` (`object`), `bool`.

Листинг:

```
s = pd.Series([10,20,30,40,50])  
print(s.dtype)
```

Результат выполнения:

int64

27. Различные типы данных в Series.

Pandas автоматически определяет тип: числа с плавающей точкой – float64, строки – object, булевы – bool.

Листинг:

```
s1 = pd.Series([1.0,2.5,3.7])  
print(s1.dtype)  
s2 = pd.Series(["apple", "banana", "cherry"])  
print(s2.dtype)  
s3 = pd.Series([True,False,True])  
print(s3.dtype)
```

Результат выполнения:

float64

object

bool

28. Преобразование типа: astype().

Метод `astype(int)` преобразует вещественные числа в целые (с отбрасыванием дробной части).

Листинг:

```
s1_int = s1.astype(int)  
print(s1_int)  
print(s1_int.dtype)
```

Результат выполнения:

0 1

```
1 2
```

```
2 3
```

```
dtype: int64
```

```
int64
```

29. Series с разнотипными данными.

При наличии разных типов (числа, строки, булевы) Pandas использует тип `object` – универсальный контейнер.

Листинг:

```
s4 = pd.Series([10, "apple", 3.14, True])
print(s4.dtype)
```

Результат выполнения:

```
object
```

30. Обнаружение пропущенных значений: `isnull()`.

Метод `isnull()` возвращает `True` для элементов, равных `NaN` или `None`.

Листинг:

```
s = pd.Series([10, np.nan, 30, None, 50])
print(s.isnull())
```

Результат выполнения:

```
0 False
```

```
1 True
```

```
2 False
```

```
3 True
```

```
4 False
```

```
dtype: bool
```

31. Обнаружение непустых значений: `notnull()`.

Метод `notnull()` – логическое отрицание `isnull()`, возвращает `True` для валидных данных.

Листинг:

```
print(s.notnull())
```

Результат выполнения:

```
0    True
```

```
1    False
```

```
2    True
```

```
3    False
```

```
4    True
```

```
dtype: bool
```

32. Фильтрация непустых значений.

Комбинация `s[s.notnull()]` возвращает Series только с валидными значениями.

Листинг:

```
filtered_s = s[s.notnull()]
```

```
print(filtered_s)
```

Результат выполнения:

```
0    10.0
```

```
2    30.0
```

```
4    50.0
```

```
dtype: float64
```

33. Выделение пропущенных значений.

Выражение `s[s.isnull()]` извлекает только элементы с пропущенными значениями.

Листинг:

```
mis_s = s[s.isnull()]
```

```
print(mis_s)
```

Результат выполнения:

```
1    NaN
```


3 NaN

dtype: float64

34. Заполнение нулями: fillna(0).

Метод fillna(0) заменяет все NaN на 0. Часто используется при подготовке данных к анализу.

Листинг:

```
s = pd.Series([10, np.nan, 30, None, 50])
s_filled = s.fillna(0)
print(s_filled)
```

Результат выполнения:

0 10.0

1 0.0

2 30.0

3 0.0

4 50.0

dtype: float64

35. Заполнение средним значением.

fillna(s.mean()) заменяет пропуски на среднее арифметическое имеющихся значений.

Листинг:

```
s = pd.Series([10, np.nan, 30, None, 50])
s_filled = s.fillna(s.mean())
print(s_filled)
```

Результат выполнения:

0 10.0

1 30.0

2 30.0

```
3  30.0
```

```
4  50.0
```

```
dtype: float64
```

36. Удаление пропущенных значений: `dropna()`.

Метод `dropna()` удаляет все строки с пропущенными значениями, возвращая очищенную Series.

Листинг:

```
s_clean = s.dropna()
```

```
print(s_clean)
```

Результат выполнения:

```
0  10.0
```

```
2  30.0
```

```
4  50.0
```

```
dtype: float64
```

37. Статистические методы.

Методы `sum()`, `mean()`, `max()`, `min()` вычисляют сумму, среднее, максимум и минимум значений.

Листинг:

```
s = pd.Series([10,20,30,40,50])
```

```
print(s.sum())
```

```
print(s.mean())
```

```
print(s.max())
```

```
print(s.min())
```

Результат выполнения:

```
150
```

```
30.0
```

```
50
```

```
10
```

38. Статистика с пропущенными значениями.

Статистические методы автоматически игнорируют NaN при вычислениях.

Листинг:

```
s_with_nan = pd.Series([10,20,None,40,50])  
print(s_with_nan.sum())  
print(s_with_nan.mean())
```

Результат выполнения:

120.0

30.0

39. Полное описание: describe().

Метод describe() возвращает статистическую сводку: count, mean, std, min, 25%, 50%, 75%, max.

Листинг:

```
print(s.describe())
```

Результат выполнения:

```
count    5.000000  
mean     30.000000  
std      15.811388  
min      10.000000  
25%      20.000000  
50%      30.000000  
75%      40.000000  
max      50.000000  
dtype: float64
```

40. describe() для строковых данных.

Для категориальных данных describe() показывает: count, unique, top (наиболее частое), freq (частота).

Листинг:

```
s_text = pd.Series(["apple", "banana", "apple", "cherry", "banana"])
print(s_text.describe())
```

Результат выполнения:

```
count      5
unique      3
top    apple
freq        2
dtype: object
```

41. Математические функции из NumPy.

Функции `np.log()`, `np.exp()`, `np.sqrt()` применяются поэлементно ко всей Series.

Листинг:

```
s = pd.Series([1,2,3,4,5])
s_log = np.log(s)
print(s_log)
print("////////")
s_exp = np.exp(s)
print(s_exp)
print("////////")
s_sqrt = np.sqrt(s)
print(s_sqrt)
```

Результат выполнения:

```
0    0.000000
1    0.693147
2    1.098612
3    1.386294
4    1.609438
```

```
dtype: float64
```

```
//////
```

```
0    2.718282
1    7.389056
2   20.085537
3   54.598150
4  148.413159
```

```
dtype: float64
```

```
//////
```

```
0    1.000000
1    1.414214
2    1.732051
3    2.000000
4    2.236068
```

```
dtype: float64
```

42. Тригонометрические функции.

Функции `np.sin()`, `np.cos()`, `np.abs()` также работают поэлементно с данными Series.

Листинг:

```
print(np.sin(s))
print(np.cos(s))
print(np.abs(s))
```

Результат выполнения:

```
0    0.841471
1    0.909297
2    0.141120
3   -0.756802
4   -0.958924
dtype: float64
```

```
0    0.540302
1   -0.416147
2   -0.989992
3   -0.653644
4    0.283662
dtype: float64
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

43. Изменение индекса.

Прямое присваивание `s.index = ['x','y','z','w']` заменяет метки индекса на новые значения.

Листинг:

```
s = pd.Series([1, 2, 3, 4], index = ['a','b','c','d'])
s.index = ['x','y','z','w']
print(s)
```

Результат выполнения:

```
x    1
y    2
z    3
w    4
dtype: int64
```

44. Сброс индекса: `reset_index()`.

Метод `reset_index()` преобразует индекс в обычный столбец, создавая новый целочисленный индекс.

Листинг:

```
s_reset = s.reset_index()
print(s_reset)
```

Результат выполнения:

```
index 0
0    x  1
1    y  2
2    z  3
3    w  4
```

45. Проверка уникальности индекса.

Свойство `is_unique` возвращает `True`, если все метки индекса уникальны, и `False` при наличии дубликатов.

Листинг:

```
s_uniq = pd.Series([10,20,30], index=['a','b','c'])
print(s_uniq.index.is_unique)

s_dupl = pd.Series([10,20,30], index=['a','b','a'])
print(s_dupl.index.is_unique)
```

Результат выполнения:

```
True
False
```

46. Доступ при дубликатах индекса.

При наличии дубликатов `s_dupl.loc['a']` возвращает все элементы с меткой 'a' в виде `Series`.

Листинг:

```
print(s_dupl.loc['a'])
```

Результат выполнения:

```
a    10
a    30
```

dtype: int64

47. Сброс индекса с удалением старого.

Параметр `drop=True` в `reset_index(drop=True)` удаляет старый индекс вместо добавления его как столбца.

Листинг:

```
s_reset = s_dupl.reset_index(drop=True)
print(s_reset)
```

Результат выполнения:

0 10

1 20

2 30

dtype: int64

48. Уникализация индекса.

Генератор списка `[f'{i}_{n}' for n, i in enumerate(s_dupl.index)]` создаёт уникальные метки, добавляя порядковый номер.

Листинг:

```
s_dupl.index = [f'{i}_{n}' for n, i in enumerate(s_dupl.index)]
print(s_dupl)
```

Результат выполнения:

a_0 10

b_1 20

a_2 30

dtype: int64

49. Сортировка по индексу.

Метод `sort_index()` сортирует элементы Series по возрастанию меток индекса.

Листинг:

```
s = pd.Series([10,20,30,40], index=['d','b','a','c'])
```



```
s_sorted = s.sort_index()
print(s_sorted)
```

Результат выполнения:

a 30

b 20

c 40

d 10

dtype: int64

50. Сортировка по индексу: по убыванию.

Параметр `ascending=False` в `sort_index()` сортирует метки индекса в обратном порядке.

Листинг:

```
s_sorted_desc = s.sort_index(ascending=False)
print(s_sorted_desc)
```

Результат выполнения:

d 10

c 40

b 20

a 30

dtype: int64

51. Сортировка по значениям.

Метод `sort_values()` сортирует элементы по возрастанию их значений, сохраняя связь с метками индекса.

Листинг:

```
s_sort_values = s.sort_values()
print(s_sort_values)
```

Результат выполнения:

d 10

b 20

a 30

c 40

dtype: int6

52. Сортировка по значениям: по убыванию.

sort_values(ascending=False) сортирует значения в порядке убывания.

Листинг:

```
s_sort_values_des = s.sort_values(ascending=False)
```

```
print(s_sort_values_des)
```

Результат выполнения:

c 40

a 30

b 20

d 10

dtype: int6

53. Сортировка с пропущенными значениями.

По умолчанию sort_values() помещает NaN в конец результата.

Листинг

```
s_nan = pd.Series([10, None, 30, None, 20], index=['a', 'b', 'c', 'd', 'e'])
```

```
print(s_nan.sort_values())
```

Результат выполнения:

a 10.0

e 20.0

c 30.0

b NaN

d NaN

dtype: float64

54. Позиция NaN при сортировке.

Параметр `na_position='first'` помещает пропущенные значения в начало отсортированного результата.

Листинг:

```
print(s_nan.sort_values(na_position='first'))
```

Результат выполнения:

```
b    NaN
d    NaN
a    10.0
e    20.0
c    30.0
dtype: float64
```

55. Временные индексы: `date_range`.

Функция `pd.date_range()` создаёт последовательность дат. При индексации по дате возвращаются соответствующие значения.

Листинг:

```
dates = pd.date_range(start='2024-03-01', periods=5, freq='D')
s = pd.Series([100,105,102,98,110], index=dates)
print(s)
print(s['2024-03-03'])
print(s['2024-03-02':'2024-03-04'])
```

Результат выполнения:

```
2024-03-01    100
2024-03-02    105
2024-03-03    102
2024-03-04     98
2024-03-05    110
Freq: D, dtype: int64
```

```
102
2024-03-02    105
2024-03-03    102
2024-03-04     98
```

Freq: D, dtype: int64

56. Часовая частота временного индекса.

Параметр `freq='h'` в `date_range()` создаёт индекс с часовым шагом.

Листинг:

```
dates = pd.date_range(start='2024-03-01', periods=5, freq='h')
s = pd.Series([10, 15, 12, 8, 20], index=dates)
print(s)
```

Результат выполнения:

```
2024-03-01 00:00:00    10
2024-03-01 01:00:00    15
2024-03-01 02:00:00    12
2024-03-01 03:00:00     8
2024-03-01 04:00:00    20
```

Freq: h, dtype: int64

57. Преобразование строк в даты.

Функция `pd.to_datetime()` преобразует строковые метки индекса в объекты `datetime` для временных операций.

Листинг:

```
s = pd.Series([10, 15, 12], index=['2024-03-01','2024-03-02','2024-03-03'])
s.index = pd.to_datetime(s.index)
print(s)
```

Результат выполнения:

```
2024-03-01    10
2024-03-02    15
```

```
2024-03-03    12
```

```
dtype: int64
```

58. Ресемплинг временных рядов.

Метод `resample('D').mean()` агрегирует данные по дням, вычисляя среднее значение для каждого периода.

Листинг:

```
print(s.resample('D').mean())
```

Результат выполнения:

```
2024-03-01    10.0
```

```
2024-03-02    15.0
```

```
2024-03-03    12.0
```

```
Freq: D, dtype: float64
```

59. Фильтрация по году.

Доступ к атрибутам времени: `s.index.year` позволяет фильтровать данные по году.

Листинг:

```
print(s[s.index.year == 2024])
```

Результат выполнения:

```
2024-03-01     10
```

```
2024-03-02     15
```

```
2024-03-03     12
```

```
dtype: int64
```

60. Сдвиг временного ряда: `shift()`.

Метод `shift(1)` сдвигает значения на 1 период вперёд, добавляя NaN в начало.

Листинг:

```
print(s.shift(1))
```

Результат выполнения:

2024-03-01 NaN

2024-03-02 10.0

2024-03-03 15.0

dtype: float64

61. Скользящее среднее: rolling().

Метод `rolling(window=3).mean()` вычисляет скользящее среднее с окном 3 значения.

Листинг

```
s = pd.Series([10,20,30,40,50,60,70], index=pd.date_range("2024-03-01",
periods=7, freq='D'))
s_ma = s.rolling(window=3).mean()
print(s_ma)
```

Результат выполнения:

2024-03-01 NaN

2024-03-02 NaN

2024-03-03 20.0

2024-03-04 30.0

2024-03-05 40.0

2024-03-06 50.0

2024-03-07 60.0

Freq: D, dtype: float64

62. Скользящее среднее с большим окном.

Увеличение окна до 5 сглаживает данные сильнее, но требует больше начальных значений (первые 4 – NaN).

Листинг:

```
s_ma5 = s.rolling(window=5).mean()
print(s_ma5)
```

Результат выполнения:

```
2024-03-01    NaN
2024-03-02    NaN
2024-03-03    NaN
2024-03-04    NaN
2024-03-05    30.0
2024-03-06    40.0
2024-03-07    50.0
```

Freq: D, dtype: float64

63. Параметр `min_periods` в `rolling()`.

`min_periods=1` позволяет вычислять среднее даже при неполном окне, уменьшая количество NaN в начале.

Листинг:

```
s_ma2 = s.rolling(window=3, min_periods=1).mean()
print(s_ma2)
```

Результат выполнения:

```
2024-03-01    10.0
2024-03-02    15.0
2024-03-03    20.0
2024-03-04    30.0
2024-03-05    40.0
2024-03-06    50.0
2024-03-07    60.0
```

Freq: D, dtype: float64

64. Визуализация скользящего среднего.

Библиотека `matplotlib` используется для построения графика исходных данных и скользящего среднего.

Листинг:

```
import matplotlib.pyplot as plt
```

```

plt.figure(figsize=(8,4))
plt.plot(s, label=r'Исходные данные', marker="o")
plt.plot(s.rolling(window=3).mean(), label=r'Скользящее среднее (3)',
linestyle='--', color='red')
plt.xlabel(r"Дата")
plt.ylabel(r"Значение")
plt.title(r"Скользящее среднее в Series")
plt.legend()
plt.grid()
plt.show()

```

Результат выполнения:

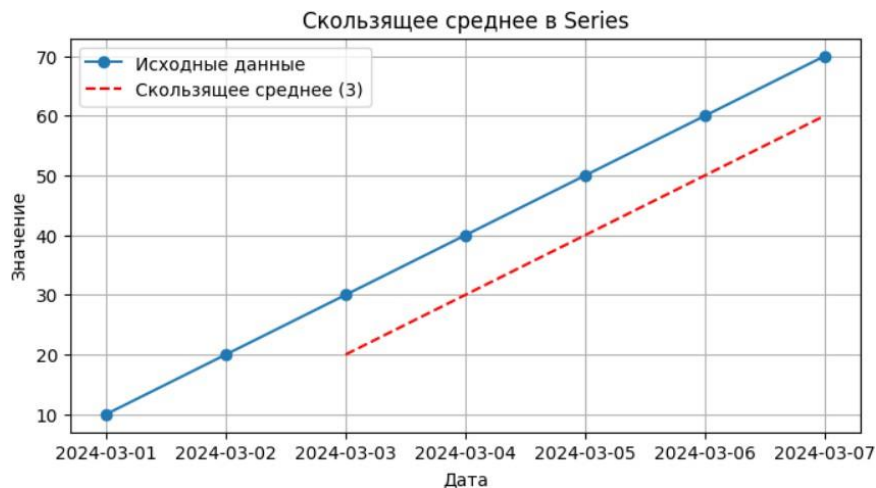


Рисунок 1 - График исходных данных и скользящего среднего

65. Процентное изменение: `pct_change()`.

Метод `pct_change()` вычисляет относительное изменение между соседними элементами: $(\text{текущее} - \text{предыдущее}) / \text{предыдущее}$.

Листинг:

```

s = pd.Series([100,110,120,90,150])
print(s.pct_change())

```

Результат выполнения:

0 NaN


```
1 0.100000
```

```
2 0.090909
```

```
3 -0.250000
```

```
4 0.666667
```

```
dtype: float64
```

66. pct_change с периодом.

Параметр periods=2 вычисляет изменение с шагом 2: сравнивает текущее значение с двумя периодами ранее.

Листинг:

```
print(s.pct_change(periods=2))
```

Результат выполнения:

```
0 NaN
```

```
1 NaN
```

```
2 0.200000
```

```
3 -0.181818
```

```
4 0.250000
```

```
dtype: float64
```

67. Визуализация временного ряда и доходности.

Построение двух графиков: цены акции и процентного изменения (доходности) с использованием matplotlib.

Листинг:

```
dates = pd.date_range(start="2024-01-01", periods=30, freq="D")
```

```
prices = pd.Series(np.random.randint(90,110, size=30), index=dates)
```

```
returns = prices.pct_change()*100
```

```
plt.figure(figsize=(10,5))
```

```
plt.plot(prices, label=r"Цена акции", marker="o")
```

```
plt.ylabel(r"Цена")
```

```
plt.xlabel(r"Дата")
```

```
plt.title(r"График цены акции")
```

```
plt.figure(figsize=(10,5))
plt.bar(returns.index, returns, color="red", alpha=0.7)
plt.axhline(0, color='black', linestyle='--')
plt.ylabel(r"Изменение (%)")
plt.xlabel(r"Дата")
plt.title(r"Процентное изменение цены акции")
plt.show()
```

Результат выполнения:



Рисунок 2 - График цены акции

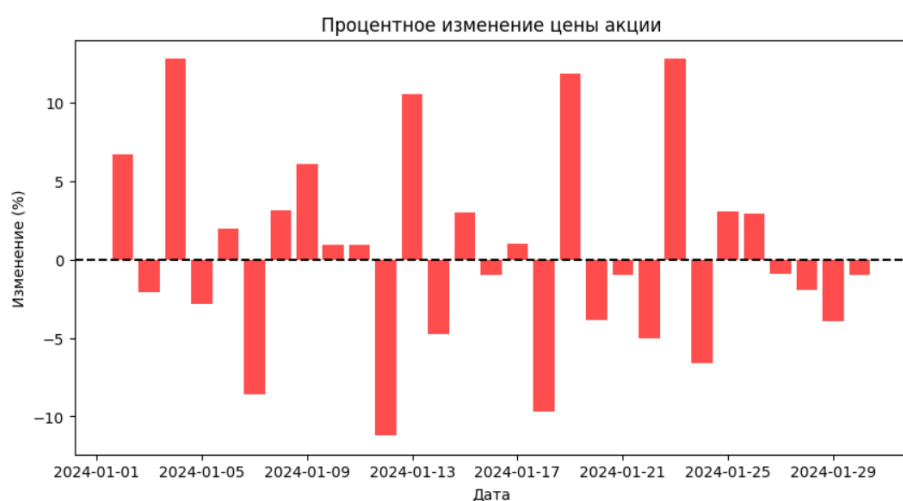


Рисунок 3 - График процентного изменения акции

Методы `to_csv()` и `read_csv()` позволяют сохранять и загружать данные Series в файл формата CSV.

Листинг:

```
data = {
    'Цена': [100, 250, 180, 320, 450],
    'Товар': ['Яблоки', 'Бананы', 'Апельсины', 'Груши', 'Виноград']
}
df = pd.DataFrame(data)
df.to_csv('data.csv', index=False, encoding='utf-8')

df = pd.read_csv("data.csv")
s = df["Цена"]
print(s)
```

Результат выполнения:

```
0    100
1    250
2    180
3    320
4    450
```

Name: Цена, dtype: int64

Задания.

Задание 1. Создание Series из списка.

Создайте Series из списка чисел [5, 15, 25, 35, 45] с индексами ['a', 'b', 'c', 'd', 'e']. Выведите его на экран и определите его тип данных.

Решение:

1. `pd.Series([5, 15, 25, 35, 45], ['a', 'b', 'c', 'd', 'e'])` создаёт Series из списка чисел с заданными индексами

2. `print(s)` выводит Series на экран
3. `s.dtype` возвращает тип данных элементов Series (в данном случае `int64` - целочисленный тип)

Листинг задания:

```
s = pd.Series([5, 15, 25, 35, 45], ['a', 'b', 'c', 'd', 'e'])  
print(s)  
print("\nТип данных:", s.dtype)
```

Результат выполнения:

```
a    5  
b   15  
c   25  
d   35  
e   45  
dtype: int64
```

Тип данных: `int64`

Задание 2. Получение элемента Series.

Дан Series с индексами ['A', 'B', 'C', 'D', 'E'] и значениями [12, 24, 36, 48, 60]. Используйте `.loc[]` для получения элемента с индексом 'C' и `.iloc[]` для получения третьего элемента.

Решение:

1. `.loc['C']` - обращается к элементу по метке индекса 'C' и возвращает значение 36.
2. `.iloc[2]` - обращается к элементу по числовой позиции 2 (третий элемент, считая с нуля) и также возвращает значение 36.

Листинг задания:

```
s = pd.Series([12, 24, 36, 48, 60], index=['A', 'B', 'C', 'D', 'E'])
```

```
print("Series:")
print(s)
element_loc = s.loc['C']
print(f"Элемент с индексом 'C': {element_loc}")
element_iloc = s.iloc[2]
print(f"Третий элемент: {element_iloc}")
```

Результат выполнения:

Series:

A 12

B 24

C 36

D 48

E 60

dtype: int64

Элемент с индексом 'C': 36

Третий элемент: 36

Задание 3. Фильтрация данных с помощью логической индексации.

Создайте Series из массива NumPy `np.array([4, 9, 16, 25, 36, 49, 64])`.

Выберите только те элементы, которые больше 20, и выведите результат.

Решение:

1. `s > 20` создает логическую маску (булев массив), которая проверяет каждый элемент на условие "больше 20"
2. `s[s > 20]` использует эту маску для фильтрации Series, оставляя только те элементы, где условие истинно (True)
3. В результате остаются элементы: 25, 36, 49 и 64 с их исходными индексами (3, 4, 5, 6)

Листинг задания:

```
s = pd.Series(np.array([4, 9, 16, 25, 36, 49, 64]))
filtered_s = s[s > 20]
print(filtered_s)
```

Результат выполнения:

```
3    25
4    36
5    49
6    64
dtype: int64
```

Задание 4. Просмотр первых и последних элементов.

Создайте Series, содержащий 50 случайных целых чисел от 1 до 100 (используйте `np.random.randint`). Выведите первые 7 и последние 5 элементов с помощью `.head()` и `.tail()`.

Решение:

1. `np.random.randint(1, 101, 50)`: генерирует массив из 50 случайных чисел.
2. `s.head(7)`: возвращает первые 7 строк Series.
3. `s.tail(5)`: возвращает последние 5 строк Series.

Листинг задания:

```
s = pd.Series(np.random.randint(1, 101, 50))
print("Первые 7 элементов:")
print(s.head(7))
print("\nПоследние 5 элементов:")
print(s.tail(5))
```

Результат выполнения:

Первые 7 элементов:

```
0    71
1    93
2    25
3    69
4    16
5    88
6    99
dtype: int32
```

Последние 5 элементов:

```
45    27
46    59
47   100
48    81
49   100
dtype: int32
```

Задание 5. Определение типа данных Series.

Создайте Series из списка ['cat', 'dog', 'rabbit', 'parrot', 'fish'].
Определите тип данных с помощью .dtype, затем преобразуйте его в category с помощью .astype().

Решение:

1. `pd.Series(...)`: создает одномерный массив (Series) из списка строк.
Изначально Pandas присваивает строковым данным тип `object`.
2. `s.dtype`: возвращает тип данных массива (в данном случае будет `object`).
3. `s.astype('category')`: преобразует тип данных в категориальный (`category`).

Листинг задания:

```
animals = ['cat', 'dog', 'rabbit', 'parrot', 'fish']
s = pd.Series(animals)
print("Исходный тип данных:", s.dtype)
s = s.astype('category')
print("Новый тип данных:", s.dtype)
print(s)
```

Результат выполнения:

Исходный тип данных: object

Новый тип данных: category

0 cat

1 dog

2 rabbit

3 parrot

4 fish

dtype: category

Categories (5, object): ['cat', 'dog', 'fish', 'parrot', 'rabbit']

Задание 6. Проверка пропущенных значений

Создайте Series с данными [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]. Напишите код, который проверяет, есть ли в Series пропущенные значения (NaN), и выведите индексы таких элементов.

Решение:

1. `pd.Series(data)`: создает объект Series из списка.

2. `s.isna()`: этот метод проверяет каждый элемент на пропуск.

Результатом будет серия булевых значений (True там, где NaN, и False там, где число).

3. `s[missing_values_mask].index`: фильтруем исходную серию, оставляя только строки с NaN, и берем их индексы.

Листинг задания:

```
data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]
s = pd.Series(data)
missing_values_mask = s.isna()
nan_indices = s[missing_values_mask].index

print("Индексы пропущенных значений (NaN):", nan_indices.tolist())
```

Результат выполнения:

Индексы пропущенных значений (NaN): [1, 3]

Задание 7. Заполнение пропущенных значений.

Используйте Series из предыдущего задания и замените все NaN на среднее значение всех непустых элементов. Выведите результат.

Решение:

1. `s.mean()` вычисляет среднее арифметическое значение всех непустых элементов Series (автоматически игнорируя NaN)
2. `s.fillna(s.mean())` заменяет все пропущенные значения (NaN) на вычисленное среднее значение
3. Результат выводится на экран с помощью `print()`

Листинг задания:

```
s = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])
mean_value = s.mean()
print(f"Среднее значение непустых элементов: {mean_value}")
s_filled = s.fillna(mean_value)

print("\nРезультат после заполнения пропусков:")
print(s_filled)
```

Результат выполнения:

Среднее значение непустых элементов: 4.25

Результат после заполнения пропусков:

0 1.20

1 4.25

2 3.40

3 4.25

4 5.60

5 6.80

dtype: float64

Задание 8. Арифметические операции с Series.

Создайте два Series:

1. `s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])`

2. `s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])`

Выполните сложение `s1 + s2`. Объясните, почему в результате появляются NaN, и замените их на 0.

Решение:

1. `s1 + s2` выполняет поэлементное сложение Series по совпадающим меткам индекса

2. NaN появляются потому, что индексы не совпадают полностью (у `s1` есть индекс 'a', которого нет в `s2`; у `s2` есть индекс 'e', которого нет в `s1`)

3. `s1.add(s2, fill_value=0)` заменяет отсутствующие значения на 0 перед выполнением операции

Листинг задания:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])
```

```

print("Series s1:")
print(s1)
print("\nSeries s2:")
print(s2)
s_sum = s1 + s2
print("\nРезультат сложения s1 + s2:")
print(s_sum)
s_sum_filled = s1.add(s2, fill_value=0)
print("\nРезультат сложения с заменой NaN на 0 (fill_value=0):")
print(s_sum_filled)

```

Результат выполнения:

```

Series s1:
a    10
b    20
c    30
d    40
dtype: int64

Series s2:
b     5
c    15
d    25
e    35
dtype: int64

Результат сложения s1 + s2:
a     NaN
b    25.0
c    45.0
d    65.0
e     NaN
dtype: float64

Результат сложения с заменой NaN на 0 (fill_value=0):
a    10.0
b    25.0
c    45.0
d    65.0
e    35.0
dtype: float64

```

Рисунок 4 - Задание 8

Задание 9. Применение функции к Series.

Создайте Series из чисел [2, 4, 6, 8, 10]. Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью `.apply(np.sqrt)`.

Решение:

1. `pd.Series([2, 4, 6, 8, 10])` создает одномерный массив с целочисленным индексом по умолчанию (0, 1, 2, 3, 4)
2. `.apply(np.sqrt)` применяет функцию квадратного корня из библиотеки NumPy к каждому элементу Series поэлементно
3. Результат возвращается в виде нового Series с сохранением исходных индексов

Листинг задания:

```
s = pd.Series([2, 4, 6, 8, 10])

print("Исходный Series:")
print(s)

s_sqrt = s.apply(np.sqrt)

print("\nРезультат применения np.sqrt (квадратный корень):")
print(s_sqrt)
```

Результат выполнения:

Исходный Series:

```
0    2
1    4
2    6
3    8
4   10
dtype: int64
```

Результат применения np.sqrt (квадратный корень):

```
0  1.414214
1  2.000000
2  2.449490
3  2.828427
4  3.162278
dtype: float64
```

Задание 10. Основные статистические методы.

Создайте Series из 20 случайных чисел от 50 до 150 (используйте np.random.randint). Найдите сумму, среднее, минимальное и максимальное значение. Выведите также стандартное отклонение.

Решение:

1. np.random.randint(50, 151, 20) генерирует 20 случайных целых чисел в диапазоне от 50 до 150 включительно (верхняя граница не включается, поэтому указано 151)
2. pd.Series() создает Series из сгенерированного массива
3. .sum(), .mean(), .min(), .max(), .std() — статистические методы, вычисляющие сумму, среднее, минимум, максимум и стандартное отклонение соответственно
4. Все методы автоматически игнорируют пропущенные значения (NaN), если таковые имеются

Листинг задания:

```
np.random.seed(42)
s = pd.Series(np.random.randint(50, 151, 20))
print("Сгенерированный Series (20 случайных чисел от 50 до 150):")
print(s)

s_sum = s.sum()
```

```
s_mean = s.mean()
```

```
s_min = s.min()
```

```
s_max = s.max()
```

```
s_std = s.std()
```

```
print("Статистические характеристики Series:")
```

```
print(f"Сумма всех элементов:      {s_sum}")
```

```
print(f"Среднее значение:          {s_mean:.2f}")
```

```
print(f"Минимальное значение:      {s_min}")
```

```
print(f"Максимальное значение:      {s_max}")
```

```
print(f"Стандартное отклонение:      {s_std:.2f}")
```

Результат выполнения:

Сгенерированный Series (20 случайных чисел от 50 до 150):

```
0    101
1    142
2     64
3    121
4    110
5     70
6    132
7    136
8    124
9    124
10   137
11   149
12    73
13    52
14    71
15   102
```

16 51

17 137

18 79

19 87

dtype: int32

Статистические характеристики Series:

Сумма всех элементов: 2062

Среднее значение: 103.10

Минимальное значение: 51

Максимальное значение: 149

Стандартное отклонение: 32.29

Задание 11. Работа с временными рядами.

Создайте Series, где индексами будут даты с 1 по 10 марта 2024 года (`pd.date_range(start='2024-03-01', periods=10, freq='D')`), а значениями - случайные числа от 10 до 100. Выберите данные за 5-8 марта.

Решение:

1. `pd.date_range(start='2024-03-01', periods=10, freq='D')` создает 10 дат: с 1 по 10 марта
2. `np.random.randint(10, 101, 10)` генерирует 10 случайных чисел от 10 до 100
3. `s['2024-03-05':'2024-03-08']` выбирает данные за указанные даты

Листинг задания:

```
s = pd.Series(  
    np.random.randint(10, 101, 10),  
    index=pd.date_range(start='2024-03-01', periods=10, freq='D')  
)  
  
print("Полный Series:")
```

```
print(s)
```

```
selected = s['2024-03-05':'2024-03-08']
```

```
print("\nДанные за 5-8 марта:")
```

```
print(selected)
```

Результат выполнения:

Полный Series:

2024-03-01	11
------------	----

2024-03-02	73
------------	----

2024-03-03	69
------------	----

2024-03-04	30
------------	----

2024-03-05	42
------------	----

2024-03-06	85
------------	----

2024-03-07	67
------------	----

2024-03-08	31
------------	----

2024-03-09	98
------------	----

2024-03-10	58
------------	----

Freq: D, dtype: int32

Данные за 5-8 марта:

2024-03-05	42
------------	----

2024-03-06	85
------------	----

2024-03-07	67
------------	----

2024-03-08	31
------------	----

Freq: D, dtype: int32

Задание 12. Проверка уникальности индексов

Создайте Series с индексами ['A', 'B', 'A', 'C', 'D', 'B'] и значениями [10, 20, 30, 40, 50, 60]. Проверьте, являются ли индексы уникальными. Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

Решение:

1. `s.index.is_unique` возвращает False, так как индексы A и B повторяются

2. `s.groupby(level=0).sum()` группирует одинаковые индексы и складывает их значения (A: 10+30=40, B: 20+60=80)

Листинг задания:

```
s = pd.Series([10, 20, 30, 40, 50, 60], index=['A', 'B', 'A', 'C', 'D', 'B'])
```

```
print("Исходный Series:")
```

```
print(s)
```

```
print(f"\nИндексы уникальны? {s.index.is_unique}")
```

```
s_new = s.groupby(level=0).sum()
```

```
print("\nРезультат после группировки:")
```

```
print(s_new)
```

Результат выполнения:

Исходный Series:

A 10

B 20

A 30

C 40

D 50

B 60

dtype: int64

Индексы уникальны? False

Результат после группировки:

A 40

B 80

C 40

D 50

dtype: int64

Задание 13. Преобразование строковых дат в DatetimeIndex.

Создайте Series, где индексами будут строки ['2024-03-10', '2024-03-11', '2024-03-12'], а значениями [100, 200, 300]. Преобразуйте индексы в DatetimeIndex и выведите тип данных индекса.

Решение:

1. `pd.Series(values, index=dates)` создает Series со строковыми индексами
2. `pd.to_datetime(s.index)` преобразует строковые индексы в объекты datetime
3. `.dtype` показывает тип данных индекса после преобразования

Листинг задания:

```
s = pd.Series([100, 200, 300], index=['2024-03-10', '2024-03-11', '2024-03-12'])
```

```
print("Исходный Series:")
```

```
print(s)
```

```
print(f"\nТип индекса до преобразования: {type(s.index)}")
```

```
s.index = pd.to_datetime(s.index)
```

```
print("\nSeries после преобразования индексов:")
print(s)
print(f"\nТип индекса после преобразования: {type(s.index)}")
print(f"Тип данных индекса: {s.index.dtype}")
```

Результат выполнения:

Исходный Series:

```
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64
```

Тип индекса до преобразования: <class 'pandas.core.indexes.base.Index'>

Series после преобразования индексов:

```
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64
```

Тип индекса после преобразования: <class 'pandas.core.indexes.dates.DatetimeIndex'>

Тип данных индекса: datetime64[ns]

Задание 14. Чтение данных из CSV-файла.

Создайте CSV-файл data.csv. Прочитайте файл и создайте Series, используя "Дата" в качестве индекса.

Решение:

1. `pd.read_csv('data.csv', index_col='Дата')` читает файл и устанавливает колонку "Дата" в качестве индекса

2. Из полученного `DataFrame` выбирается столбец "Цена" для создания `Series`

Листинг задания:

```
data = {
    'Дата': ['2024-03-01', '2024-03-02', '2024-03-03', '2024-03-04', '2024-03-05'],
    'Цена': [100, 110, 105, 120, 115]
}
df = pd.DataFrame(data)
df.to_csv('data.csv', index=False, encoding='utf-8')
print("Файл создан!")
s = pd.read_csv('data.csv', index_col='Дата')['Цена']

print("\nПолученный Series:")
print(s)
```

Результат выполнения:

Файл создан!

Полученный Series:

Дата

2024-03-01 100

2024-03-02 110

2024-03-03 105

2024-03-04 120

2024-03-05 115

Name: Цена, dtype: int64

Задание 15. Построение графика на основе Series

Создайте Series, где индексами будут даты с 1 по 30 марта 2024 года, а значениями – случайные числа от 50 до 150. Постройте график значений с помощью matplotlib. Добавьте заголовок, подписи осей и сетку.

Решение:

1. `pd.date_range('2024-03-01', periods=30, freq='D')` создает 30 дат с 1 по 30 марта
2. `np.random.randint(50, 151, 30)` генерирует 30 случайных чисел от 50 до 150
3. `plt.plot()` строит график, где по оси X — даты, по оси Y — значения
4. Для использования LaTeX в подписях перед строкой ставится `r` и формулы заключаются в `$...$`
5. `plt.title()`, `plt.xlabel()`, `plt.ylabel()` добавляют заголовок и подписи осей с LaTeX
6. `plt.grid(True)` включает сетку

Листинг задания:

```
import matplotlib.pyplot as plt
```

```
s = pd.Series(  
    np.random.randint(50, 151, 30),  
    index=pd.date_range('2024-03-01', periods=30, freq='D')  
)
```

```
plt.plot(s)  
plt.title(r'График функции  $y = f(x)$  за март 2024 г.')  
plt.xlabel(r'Дата ( $x$ )')  
plt.ylabel(r'Значение ( $y$ )')  
plt.grid(True)
```

plt.show()

Результат выполнения:

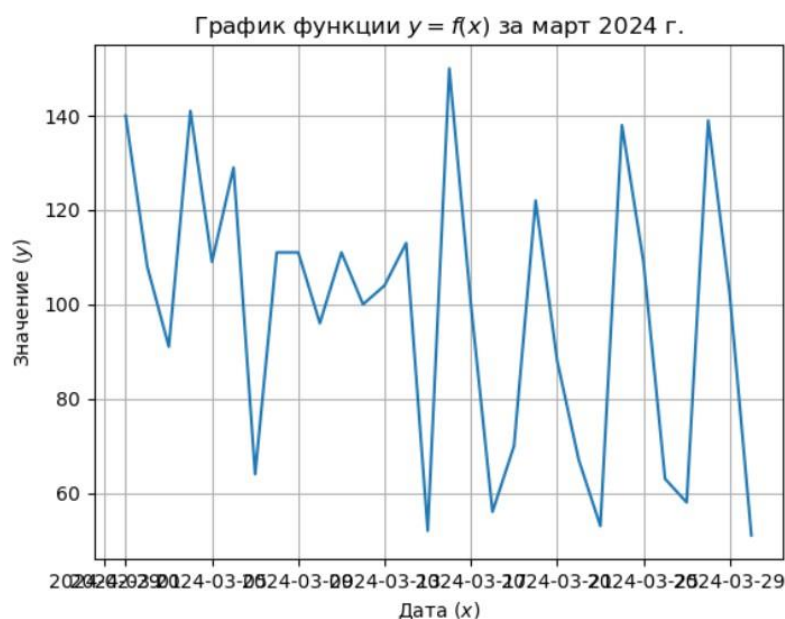


Рисунок 5 - График для задания 15

Индивидуальные задания.

Задание 1. Анализ энергопотребления.

Создайте Series, где индексами будут часы суток (`pd.date_range('2024-05-10', periods=24, freq='H')`), а значениями – случайное потребление электроэнергии в кВт-ч (от 1 до 5).

Преобразуйте индекс в `DatetimeIndex` и постройте график, выделив цветом периоды максимального потребления (больше 4 кВт-ч).

Решение:

1. `pd.date_range('2024-05-10', periods=24, freq='H')` создает 24 часа с 00:00 до 23:00 10 мая 2024 года
2. `np.random.uniform(1, 5, 24)` генерирует 24 случайных числа от 1 до 5
3. Индекс автоматически является `DatetimeIndex` при создании через `date_range`

4. Для выделения областей с потреблением > 4 кВт·ч используется `plt.fill_between()` с условием `where`

5. `plt.plot()` строит основной график

Листинг задания:

```
s = pd.Series(
    np.random.uniform(1, 5, 24),
    index=pd.date_range('2024-05-10', periods=24, freq='H')
)

plt.figure(figsize=(12, 5))
plt.plot(s, 'b-o', label=r'$P(t)$')
plt.fill_between(s.index, s.values, where=(s.values > 4), color='red',
alpha=0.4,
                label=r'$P(t) > 4$')
plt.axhline(y=4, color='orange', linestyle='--', label=r'$P_{\text{порог}} = 4$')

plt.title(r'График энергопотребления $P(t)$ за сутки')
plt.xlabel(r'Время $t$ (часы)')
plt.ylabel(r'Потребление $P$ (кВт$\cdot$ч)')
plt.grid(True)
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Результат выполнения:



Рисунок 6 - График для индивидуального задания

Контрольные вопросы:

1. Что такое `pandas.Series` и чем она отличается от списка в Python?

`Series` — это одномерный массив данных с метками, которые называются индексами. Отличие от списка в том, что каждый элемент `Series` имеет свою метку (индекс), тогда как в списке элементы доступны только по числовой позиции.

2. Какие типы данных можно использовать для создания `Series`?

Целые и дробные числа, строки, логические значения `True` и `False`, даты и время, а также массивы `NumPy`.

3. Как задать индексы при создании `Series`?

При создании `Series` нужно передать параметр `index` со списком меток. Например: `pd.Series([10, 20, 30], index=['a', 'b', 'c'])`.

4. Каким образом можно обратиться к элементу `Series` по его индексу?

Нужно написать имя `Series` и в квадратных скобках указать метку индекса. Например: `s['a']`.

5. В чём разница между `.iloc[]` и `.loc[]` при индексации `Series`?

`.loc[]` работает с метками индексов, а `.iloc[]` — с числовыми позициями. `.loc['a']` вернёт элемент с меткой 'a', а `.iloc[0]` вернёт первый элемент независимо от его метки.

6. Как использовать логическую индексацию в `Series`?

Нужно внутри квадратных скобок написать условие. Например: `s[s > 10]` вернёт только те элементы, которые больше 10.

7. Какие методы можно использовать для просмотра первых и последних элементов `Series`?

Метод `.head()` показывает первые несколько элементов, `.tail()` —

последние. По умолчанию выводятся 5 элементов, но можно указать любое число.

8. Как проверить тип данных элементов Series?

Нужно использовать свойство `.dtype`. Например: `s.dtype` покажет тип данных всех элементов Series.

9. Каким способом можно изменить тип данных Series?

Нужно использовать метод `.astype()`. Например: `s.astype('float')` превратит целые числа в дробные.

10. Как проверить наличие пропущенных значений в Series?

Можно использовать методы `.isna()` или `.isnull()`. Они возвращают `True` там, где есть пропуски. Метод `.hasnans` показывает, есть ли хотя бы один пропуск.

11. Какие методы используются для заполнения пропущенных значений в Series?

Основной метод — `.fillna()`. Он заменяет пропуски на указанное значение. Например, `s.fillna(0)` заменит все пропуски на ноль.

12. Чем отличается метод `.fillna()` от `.dropna()`?

`.fillna()` заменяет пропущенные значения на другое значение, сохраняя все элементы. `.dropna()` полностью удаляет строки с пропущенными значениями.

13. Какие математические операции можно выполнять с Series?

Можно складывать, вычитать, умножать, делить Series друг на друга или на обычные числа. Также доступны возведение в степень, вычисление квадратного корня и другие операции.

14. В чём преимущество векторизованных операций по сравнению с циклами Python?

Векторизованные операции работают сразу со всеми элементами одновременно, что делает код короче и значительно быстрее, особенно на больших данных. Циклы обрабатывают элементы по одному и работают медленнее.

15. Как применять пользовательскую функцию к каждому элементу Series?

Нужно использовать метод `.apply()`. Например, `s.apply(lambda x: x * 2)` умножит каждый элемент на 2.

16. Какие агрегирующие функции доступны в Series?

`.sum()` — сумма, `.mean()` — среднее, `.min()` — минимум, `.max()` — максимум, `.count()` — количество элементов, `.std()` — стандартное отклонение.

17. Как узнать минимальное, максимальное, среднее и стандартное отклонение Series?

Используйте методы: `s.min()`, `s.max()`, `s.mean()`, `s.std()`.

18. Как сортировать Series по значениям и по индексам?

`.sort_values()` сортирует по значениям, `.sort_index()` — по индексам.

19. Как проверить, являются ли индексы Series уникальными?

Нужно использовать свойство `.index.is_unique`. Оно вернёт `True`, если все индексы разные, и `False`, если есть повторяющиеся.

20. Как сбросить индексы Series и сделать их числовыми?

Нужно использовать метод `.reset_index()`. Он превращает текущие индексы в обычный столбец и создаёт новые числовые индексы от 0.

21. Как можно задать новый индекс в Series?

Нужно присвоить новый список меток свойству `.index`. Например: `s.index = ['x', 'y', 'z']`.

22. Как работать с временными рядами в Series?

Временные ряды создаются с помощью `pd.date_range()` для генерации дат. Затем эти даты передаются как индекс Series. После этого можно выбирать данные по датам, использовать срезы по датам и применять специальные методы для работы с временными данными.

23. Как преобразовать строковые даты в формат DatetimeIndex?

Нужно использовать `pd.to_datetime()`. Например: `s.index = pd.to_datetime(s.index)`. Это превращает строки с датами в специальный тип данных для работы с датами.

24. Каким образом можно выбрать данные за определённый временной диапазон?

Можно использовать срез по датам: `s['2024-03-01':'2024-03-10']` выберет все данные с 1 по 10 марта.

25. Как загрузить данные из CSV-файла в Series?

Сначала читаем весь файл через `pd.read_csv()` в DataFrame, затем выбираем нужный столбец. Например: `df = pd.read_csv('file.csv'); s = df['название_столбца']`.

26. Как установить один из столбцов CSV-файла в качестве индекса Series?

При чтении файла можно использовать параметр `index_col`: `df = pd.read_csv('file.csv', index_col='название_столбца')`. Затем извлечь нужный столбец в Series.

27. Для чего используется метод `.rolling().mean()` в Series?

Этот метод вычисляет скользящее среднее. Он берёт окно из нескольких

элементов и считает среднее по этому окну, затем окно сдвигается. Это помогает сгладить колебания данных и увидеть общий тренд.

28. Как работает метод `.pct_change()`? Какие задачи он решает?

`.pct_change()` вычисляет процентное изменение между текущим и предыдущим элементом. Он показывает, на сколько процентов выросло или упало значение по сравнению с прошлым периодом. Это полезно для расчёта доходности или темпов роста.

29. В каких ситуациях полезно использовать `.rolling()` и `.pct_change()`?

`.rolling()` полезен для сглаживания шумов на графиках и поиска трендов в данных. `.pct_change()` полезен для расчёта доходности акций, темпов роста продаж, инфляции и любых других данных, где важно относительное изменение.

30. Почему NaN могут появляться в Series, и как с ними работать?

NaN появляются, когда при сложении двух Series индексы не совпадают, при чтении данных с пустыми ячейками в файле или при делении на ноль. Работать с ними можно так: `.isna()` — для поиска пропусков, `.fillna()` — для замены пропусков на нужное значение, `.dropna()` — для удаления строк с пропусками.

Вывод:

В ходе выполнения данной лабораторной работы были изучены основы работы с библиотекой `pandas`, в частности со структурой данных Series. В процессе работы были освоены различные способы создания Series: из списка, из массива NumPy, из словаря, а также со скалярным значением. Были изучены методы доступа к элементам Series с помощью меток (`loc`) и целочисленных позиций (`iloc`), а также применение срезов и логической индексации для фильтрации данных. Рассмотрены арифметические операции с Series, включая сложение с обработкой несовпадающих индексов через параметр `fill_value`. Особое внимание уделено работе с пропущенными

значениями: их обнаружению методами `isnull()` и `notnull()`, заполнению через `fillna()` (нулями или средним значением) и удалению через `dropna()`. Также были изучены статистические методы `Series` (`sum`, `mean`, `min`, `max`, `std`, `describe`), преобразование типов данных с помощью `astype()`, сортировка по индексам и значениям, проверка уникальности индексов и работа с дубликатами. Были рассмотрены способы изменения и сброса индексов, а также преобразование строковых дат в формат `DatetimeIndex` для работы с временными рядами, что включает создание временного индекса с помощью `date_range`, фильтрацию данных по датам, ресемплинг, сдвиг значений, расчёт скользящего среднего и процентного изменения. На практике был реализован анализ энергопотребления с визуализацией графика и выделением областей максимального потребления.